



THE WONDERS OF SVANETI

Module Creative-Yumland — Développement Web PHP

Rapport de Projet — Phase 3

Membres du groupe	Module	Date de rendu
Wael HAJJI	Creative-Yumland	17 mai 2026
Nidal MEKEDDEM	Développement Web PHP	Année 2024-2025

SECTION 01

Présentation du groupe

Notre groupe est toujours composé des deux mêmes membres. Pour cette phase, on s'est réparti les tâches de façon assez naturelle : Wael a géré la réflexion sur ce qu'on devait rendre interactif et comment l'organiser, Nidal a pris en charge le code JavaScript et PHP côté traitement. Dans les faits, on s'est beaucoup entraïdés parce que JavaScript était une vraie nouveauté pour nous deux.

Wael HAJJIwael.hajji@etu.cyu.fr

Organisation des fonctionnalités à rendre dynamiques, tests dans le navigateur, validation du rendu final et commit sur GitHub.

Nidal MEKEDDEMnidal.mekeddem@etu.cyu.fr

Écriture du JavaScript et des fichiers PHP associés, débogage des interactions, intégration dans les pages existantes.

SECTION 02

Organisation et planning

Tout le développement a été réalisé sur Visual Studio Code avec un serveur local XAMPP pour tester en direct dans le navigateur. Une fois l'ensemble des fonctionnalités validées, on a fait le commit final et poussé le projet sur GitHub.

Semaine 1 — Découverte de JavaScript et modification du profil**27 avril
2026**

- Lecture du sujet et compréhension de ce qu'on devait faire avec JavaScript
- Prise en main de `fetch()` et de `FormData` pour envoyer des données sans recharger la page
- Modification du profil utilisateur : création de `modifier_profil.php` et `modification_profil.js`

- Mise en place du thème sombre avec `theme.js` et mémorisation via cookie

Semaine 2 — Filtres, administration et commandes

10 mai 2026

- Filtres produits sans rechargement et autocomplete sur la barre de recherche
- Bloquer / débloquent un utilisateur depuis la page admin
- Changement de statut des commandes en temps réel pour le restaurateur
- Annulation de commande sans rechargement pour le client
- Tests complets dans le navigateur, puis commit et push sur GitHub

SECTION 03

Problèmes rencontrés

1 Les boutons qu'on ajoute après coup n'écotent rien

Sur la page commandes, quand on clique sur "En préparation", on insère une nouvelle ligne avec un nouveau bouton. Le problème : ce bouton n'avait aucun comportement. On avait mis les listeners au chargement de la page, donc tout ce qui arrivait après était ignoré. On a mis du temps à comprendre pourquoi le deuxième clic ne faisait rien.

2 Faire passer des données PHP vers JavaScript

Pour construire dynamiquement le formulaire d'affectation de livreur, il nous fallait la liste des livreurs en JavaScript. Mais cette liste vient de PHP et une fois la page chargée, on ne peut plus y accéder directement. On ne savait pas du tout comment faire passer cette information côté JavaScript.

3 La ligne disparaît mais n'apparaît nulle part

Quand on passait une commande au statut suivant, on masquait la ligne — mais on ne la recréait pas dans la bonne section. La commande semblait disparaître comme si elle avait été supprimée. Il fallait lire les données de la ligne avant de la cacher, construire le HTML de la nouvelle ligne à la main, et l'insérer dans la bonne section.

4 Deux listes de suggestions qui s'affichent en même temps

Quand on a ajouté notre propre liste de suggestions sur la barre de recherche, le navigateur affichait aussi sa propre liste automatique juste en dessous. Résultat : deux menus déroulants en même temps, c'était impossible à utiliser.

SECTION 04

Solutions apportées

1 On a mis un seul listener sur le conteneur parent de toute la page commandes. Chaque clic remonte jusqu'à lui, et on vérifie si l'élément cliqué est bien un bouton de changement de statut. Comme ça, que le bouton soit là dès le départ ou ajouté 10 secondes plus tard, il fonctionne de la même façon.

2 On a écrit la liste des livreurs directement dans la page HTML via une balise `<script>` générée par PHP : `var livreurs_data = <?php echo json_encode(...); ?>`. JavaScript peut alors lire cette variable comme n'importe quelle autre, et s'en servir pour construire le formulaire dynamiquement.

3 Avant de cacher la ligne, on récupère le contenu de chaque cellule (date, résumé, total). On construit ensuite le HTML de la nouvelle ligne à partir de ces données et on l'insère avec `insertAdjacentHTML("beforeend", ...)` dans la bonne section. On met aussi à jour les compteurs affichés en haut de chaque colonne.

4 Il suffisait d'ajouter `autocomplete="off"` sur le champ de recherche. Cet attribut dit au navigateur de ne pas afficher sa propre liste de suggestions, ce qui laisse uniquement la nôtre apparaître.

SECTION 05

Bilan

La Phase 3 a été celle où on a le plus appris en peu de temps. JavaScript, on en avait entendu parler, mais l'appliquer concrètement sur un projet qu'on avait construit soi-même, c'est différent. Le plus dur a été de comprendre que le code ne s'arrête pas pour attendre la réponse du serveur — il faut tout gérer dans la suite du `fetch()`.

Tout le développement s'est fait sur Visual Studio Code, avec XAMPP pour tester en local et le navigateur toujours ouvert à côté. Une fois tout validé fonctionnellement, on a fait le commit final et poussé le projet sur GitHub.

Toutes les fonctionnalités demandées sont présentes et fonctionnent correctement.

Fonctionnalités livrées

- ✓ Modification du profil sans rechargement (6 champs)
- ✓ Filtres produits dynamiques sans rechargement
- ✓ Autocomplete sur la barre de recherche
- ✓ Admin : bloquer / débloquer un utilisateur
- ✓ Restaurateur : changement de statut en temps réel
- ✓ Client : annulation de commande
- ✓ Blocage à la connexion pour les comptes suspendus
- ✓ Thème sombre persistant via cookie